Field Hours

CSCE 465 Computer and Network Security

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 50 minutes

A threat, named

A threat is an actor, a capability, and an asset. 'Hackers' is not a threat model. 'Someone who can send packets to the clinic server, and wants another patient's record' is one. Write three. If the model is empty, every later control is a guess.

You will not stop every actor. Say which one this course's mechanisms address. CSCE 701 builds on a model like this. It will not spend its first week forcing you to name the actor.

One threat, three lines.

Who: a person on the public network

Can: send requests, not read the disk

Wants: one patient's record

The clinic, three threats

1. A remote requester who is not staff, sending ordinary requests.

2. A staff account that looks up patients outside their team. The actor is inside.

3. Someone who can read a stolen backup. They do not need a login if the backup is the record.

4. The controls differ. A firewall does nothing about the third.

Problem 1. The product has a firewall and you claim it is secure. Which of the three threats did you ignore?

Answer. At least the stolen backup, and the staff member who is allowed through the firewall. A firewall is one control for one capability. Name the threat it does not touch.

Problem 2. Why is this course on the site if CSCE 701's catalog line only says graduate classification?

Answer. Graduate classification assumes an undergraduate security course. CSCE 465 is that course. Take it, and CSCE 313, before 701. The hours sit outside the 36 credits.

Lecture 2 · 50 minutes

Memory

A buffer has a length. Writing past it changes adjacent memory. On a stack, that memory can include a saved return address. The program then jumps where the extra bytes said. That is the classic overflow. The fix is to refuse the extra bytes, and to make the return address harder to forge. You need the layout from CSCE 312 and

the pointer from CSCE 313 before this sentence is obvious.

A format string, or a use of memory after it was freed, is the same family: the program treats attacker bytes as a meaning the programmer did not intend. Find the place where the size is trusted.

A stack frame, simplified.

Local buffer

Saved frame

Saved return address

Bytes past the buffer walk upward into the return

Eight bytes of room

1. The buffer holds 8 bytes. The input is 16. The copy does not check.

2. The second 8 bytes land on the saved return address. The function returns into those bytes.

3. Check the length and stop at 8, or refuse the call. The graduate course will add the surrounding properties. The check is this course.

Problem 1. The language checks the length for you. Are you done with this bug?

Answer. For that buffer, in that language, the overrun is no longer yours to forget. Other bugs remain: a logic check that uses the wrong identity, a secret in a log. Do not import a C habit into a safe language, and do not assume the safe language ended security.

Problem 2. You made the stack non-executable. The overflow still happens. What did you stop, and what did you not?

Answer. You stopped the easy case of running the bytes that were written onto the stack. You did not stop the overwrite. A return into existing code can still be enough. The length check is still required.

Lecture 3 · 50 minutes

A secret and a hash

A password is a secret the user can remember. You do not store it. You store a slow hash of the password plus a salt. The salt is unique per user so two users with the same password do not look the same in the file. A fast hash is wrong here, because guessing is the attack.

Encryption is a different tool. It hides a message from someone without the key. Hashing a password is not encryption. You cannot decrypt a hash. Systems that 'decrypt the password to email it back' were storing the password.

Two tools.

Slow salted hash

Store this for a password

No need to recover the text

Encryption

Hide a message you must read later

Wrong tool for a password database

Two users, one password

1. Ada and Ben both choose the same passphrase.

2. Ada's salt is S1, Ben's is S2. The stored values differ, because the hash inputs differ.

3. A stolen file does not show that they match, and a precomputed table of unsalted hashes does not apply to either row.

Problem 1. You hashed passwords with a fast checksum and no salt. What is the attack?

Answer. Guess common passwords, hash them once, and compare against every row. One computation per guess covers the whole file. A salt forces the guess to be redone per row. A slow hash makes each guess expensive. Use both.

Problem 2. The product emails the user their current password. What does that imply about storage?

Answer. The password is recoverable, so it is not only a slow hash. It is stored in a reversible form, or in the clear. Replace that with a reset token. You should not be able to read the password back.

Lecture 4 · 50 minutes

The network attacker

On an open path, the attacker can read, drop, replay, or change packets. A password sent in the clear is then the attacker's password. A lock icon is not the analysis. The analysis is: which bytes were secret, and who else could see them.

A replay sends an old correct message again. If the server accepts it, the attacker does not need the secret, only a copy of the request. A nonce, a timestamp, or a one-time token is how the server notices the copy. CSCE 701 will formalize this. You should already refuse a design that replays.

A replay.

1. The real client sends a valid request

2. The attacker stores it

3. The attacker sends the same bytes later

4. The server must be able to say: I have seen this

A transfer request

1. The message says: move ten dollars from A to B, with a valid signature.

2. The attacker cannot forge a new signature. They can send the same signed message tomorrow.

3. If the server has no memory of that message, it moves the money twice. Store a unique id from the

message and reject the second one, or bind the signature to a one-time challenge.

Problem 1. The channel is encrypted and you still accept a replayed ciphertext. What was missing?

Answer. Freshness. Encryption hides the bytes from a reader. It does not, by itself, tell you this ciphertext is a second copy of an old one, unless the protocol binds a nonce that the server checks. Turn the encryption on and still check the replay.

Problem 2. You log the full request, including the password, 'for debugging'. Who is the new attacker?

Answer. Anyone who can read the log: an operator, a stolen backup of the logs, a too-wide admin tool. The threat model grew. Do not write secrets into logs. Log an id, not the secret.

Lecture 5 · 50 minutes

Who may do what

Authentication says who is calling. Authorization says what that identity may do. A checked password is not a permission. The permission is a rule: this identity may read this row. Check it on the server, on every request. A hidden button is not a check. The request can be sent without the button.

A confused deputy is a program that has a privilege and uses it on behalf of a caller who does not. The clinic server can read every row. The clerk may not. The server must use the clerk's limit, not its own, when answering the clerk. CSCE 701 names this. You should already see it in the query.

Two questions.

Authentication

This request is Ada

A password, a key, a session

Authorization

Ada may see this row

Checked even if the button is hidden

Another patient's id

1. Ada is authenticated. She changes the id in the request from her patient to Ben's.

2. The server looks up Ben's row because the database account can see every row.

3. The missing check is: is Ada on Ben's care team? If not, return nothing. The hidden button on Ada's page never ran. The request did.

Problem 1. The page hides the button for other patients. Is the record safe?

Answer. No. Hiding a button is a display choice. The request is the authority boundary. Check the team on the server for every id. Assume the caller can send any id.

Problem 2. The server process runs as a database user that can read all rows. Is that a bug by itself?

Answer. Not always. The process needs a way to read the rows it is allowed to serve. The bug is using that power for a caller who was not allowed. Constrain the query with the caller's identity. That is the deputy.

Lecture 6 · 50 minutes

What the graduate course will assume

You can write a threat in three lines, explain an overflow as a length that was not checked, store a password as a slow salted hash, spot a replay, and separate a hidden button from a server-side check. CSCE 701 will talk about properties, cryptography as a tool, and incidents. It should not be the first time you see these.

CSCE 313 is the other preparation: processes, memory, and files. Take that too if the machine underneath these attacks is still vague.

Five checks before 701.

A named threat

A length check

A salted slow hash

A replay the server refuses

A permission checked on the server

A question you should answer cold

1. The firewall is on, the password is hashed, and a staff member exports every patient. Which control failed?

2. Authorization. The staff identity was real. The export was not limited to their team. The firewall and the hash addressed other threats.

3. If you blamed the firewall, redo the threat lecture before the graduate course.

Problem 1. Name two courses to finish before CSCE 701, and why neither is inside the 36.

Answer. CSCE 313 for the machine, and CSCE 465 for threats and the basic mechanisms. Both are undergraduate preparation. 701 is the master's course and is the one that counts toward the ten.

Problem 2. A test that used to overflow now stops at the buffer length. What do you keep?

Answer. The test. It is the regression that the length check stays. Deleting it because the bug is fixed is how the next edit removes the check.